

Onyx — the NetSurf changes

Onyx · multi-process OS on Raspberry Pi 4

NetSurf is Onyx's web browser: the upstream NetSurf 3.x core, its framebuffer frontend and its libraries, vendored in `third_party/` and changed for Onyx. This document lists what differs from upstream and why, as `05-CIRCLE-CHANGES.md` does for Circle. The aim of the changes: pages drawn as Chrome draws them (the user compares with Chrome on Windows).

NetSurf is ours to change: the patches are marked `Onyx:` in the sources (a comment on the line or the block), and each is listed here. The user's guide entry is in `04-USER-GUIDE.md` (NetSurf), the build in `user/netsurf/README.md`.

1. Where things are

Path	What
<code>third_party/netsurf/</code>	NetSurf: <code>content/</code> (the core: HTML, CSS, layout, redraw), <code>desktop/</code> , <code>frontends/framebuffer/</code> (the frontend Onyx uses), <code>include/</code> , <code>utils/</code>
<code>third_party/libcss/</code>	CSS parser and selection engine (most of the CSS3 work)
<code>third_party/libnsfb/</code>	the framebuffer library (<code>user/nsfb/onyx_surface.c</code> : its Onyx surface)
<code>third_party/freetype-2.14.3/</code>	FreeType (options and modules: <code>user/netsurf/freetype/</code>)
<code>third_party/brotli-1.1.0/</code>	Brotli's decoder only (FreeType's WOFF2), MIT
<code>third_party/quickjs-ng-0.17.0/</code>	QuickJS-ng, the JavaScript engine (ES2023), MIT: the engine alone (<code>README.onyx</code> : its patches)
<code>third_party/netsurf/content/handlers/javascript/quickjs/</code>	NetSurf's JavaScript on QuickJS: <code>qjs.c</code> (the engine's glue, the natives), <code>dom.js</code> (the DOM, in JavaScript)
<code>third_party/fonts/</code> , <code>third_party/dejavu-fonts-ttf-2.37/</code>	the fonts staged into <code>SD:/res/fonts</code>
<code>user/netsurf/</code>	the Onyx glue: <code>onyx_chrome.cpp</code> (the window, its wtk toolbar, the History dialog), <code>onyx_fetch.c</code> (HTTP/HTTPS over the Onyx TCP kapis, mbedTLS; each download in a thread of its own), <code>onyx_main.c</code> , the makefiles
<code>tools/tests/netsurf/</code>	the PC test bench: NetSurf built for the PC on the desktop simulator (<code>host.mk</code>),

Path	What
	a page to a PNG (<code>shot.sh</code>), the same page in Chromium (<code>chrome.sh</code>), copies of the two sites (<code>getsites.sh</code>), the JavaScript regression test (<code>jstest.sh</code> , <code>pages/js-*.html</code>); <code>NS_JSDEBUG=1</code> prints the scripts' errors and <code>console.log</code> , <code>NS_BOXDUMP=<file></code> + F5 dumps the box tree. <code>jstest.sh</code> : the DOM, the events, a recursion, <code>fetch</code> / XHR (file:// and data: URLs), the hover events, CSS <code>:hover</code> , <code>localStorage</code> kept

Build for the Pi: `make -C user/netsurf` (the libraries, their `.a` are committed: `libquickjs.a` among them), then `make -f user/netsurf/netsurf-app.mk link stage`. A header change needs a clean rebuild of what includes it (the rules do not track headers): the libcss objects, `/tmp/nsbuild`. The codegen tools need a host compiler (`BUILD_CC`, `gcc` by default: `build-essential` in WSL; a WSL without one: `BUILD_CC=$PWD/tools/wsl-build-cc.sh`, the Windows MinGW `gcc`). GCC 14 turned some warnings NetSurf has into errors: the makefile keeps them warnings.

The network (`user/netsurf/onyx_fetch.c`): each download runs in a **thread** of its own (kernel v67) -- the DNS, the connect, the TLS handshake and every read block there, not the UI; the UI thread gets the finished ones in the fetcher's poll (every 10 ms) and hands them to NetSurf (the head parsed, the redirects, the inflate, the callbacks). 8 at once; an aborted fetch whose thread still runs is freed by that thread. The request is a GET, a POST (a url-encoded or text body) or any method a script asks for (`X-Onyx-Method`, taken off), with the caller's headers; the response goes to the core with its status line and headers (not the cache-control ones, nor those the inflate makes wrong). Without threads (an older kernel, the PC bench) it is the one-step-per-poll state machine. mbedTLS's session cache is locked (`onyx_tls.hpp`). The cache (`content/llcache.c`, Onyx changes): a request's own headers (`llcache_handle_retrieve_ex`), its HTTP status kept (`llcache_handle_get_http_code`), its headers in order (`llcache_handle_get_header_at`).

2. The window and the frontend

- **A native Onyx window** (`user/netsurf/onyx_chrome.cpp`): the toolbar is wtk's (back, forward, reload/stop, home, History, the address field), the frame's close box closes NetSurf; no fbtck toolbar (`frontends/framebuffer/gui.c`). Keys: F5, Esc, Alt+Left/Right, F6 / Ctrl+L, Ctrl+H.
- **History** — a native dialog (most recent first, Find, Delete, Clear all); the pages visited kept in `SD:/apps/netsurf.app/History`, the cookies in `.../Cookies` (`ONYX_NS_DATAPATH`, written a few seconds after each page and on exit; never committed). The local history's thumbnails are off (`desktop/browser_history.c`).
- **Scrolling**: the mouse wheel; a scroll blits what stays on screen and redraws the strip that comes in. A new page opens at its top (`GW_EVENT_NEW_CONTENT`).
- **96 dpi, 16 px text** (`gui.c`): a CSS px is a pixel, `medium` is 12 pt = 16 px, the default font is serif (Chrome's Times New Roman) — NetSurf had 90 dpi and 12.8 pt.

3. Fonts (`frontends/framebuffer/font_freetype.c`)

- **Families by their CSS names.** NetSurf used only the generic family. The card's fonts (`SD:/res/fonts`) are found under their names and those of the fonts they stand in for, metric for metric (`fb_card_faces`, `fb_font_aliases`):

font on the card	stands in for
Liberation Sans 2.1.5	Arial, Helvetica, Arimo, Tahoma, Trebuchet MS
Liberation Serif 2.1.5	Times New Roman, Times, Tinos
Selawik 1.01 (Microsoft)	Segoe UI, system-ui
Gelasio	Georgia (and Georgia's ascent / descent)
DejaVu Sans / Serif / Sans Mono	Verdana, Courier New, Consolas, Menlo... — and the fallback

The generic families are Chrome's on Windows: serif → Liberation Serif, sans-serif → Liberation Sans, monospace → DejaVu Sans Mono. All are SIL OFL; a missing file is skipped (DejaVu Sans is the one required).

- **Matching** as CSS Fonts says: the style first (an italic with no italic face is its roman slanted, $x \neq y/4$), then the nearest weight (600 and up is bold with two faces).
- **Per-character fallback:** the families of the list in turn, then DejaVu Sans.
- **Advances:** unhinted (fractional), each glyph drawn at its pen position rounded; glyphs hinted lightly (`FT_LOAD_TARGET_LIGHT`, vertically only). `letter-spacing`, `word-spacing` (new `plot_font_style_t` fields) and `font-variant: small-caps` apply.
- **Vertical metrics** (`fb_font_metrics`, the layout table's `metrics`): as DirectWrite reads them — OS/2's typographic metrics when the font sets `USE TYPO METRICS`, else its Windows ascent / descent and `hhea`'s line gap; each rounded to whole pixels as Blink does.
- **Web fonts** (`@font-face`, `content/handlers/html/onyx_webfont.c`): once a document's style sheets are in, each face's first source FreeType reads (TrueType, OpenType/CFF, WOFF, WOFF2) is fetched, handed to the font code for that document only (`add_face` / `release_faces` / `set_scope` in the layout table) and the document laid out again (`font-display: swap`), its cached text widths forgotten first. Faces split by `unicode-range` are fetched when they cover some of Latin-1; a variable font is set to the weight asked (its `wght` axis). `onyx_fetch.c` asks `fonts.googleapis.com` with a current browser's user agent (WOFF2 subsets, ~50 KB a weight, rather than 300 KB TTFs).
- FreeType is built with the TrueType and CFF drivers, zlib (WOFF) and Brotli (WOFF2).

4. CSS (`libcss`)

- **Values:** `calc()`, `min()`, `max()`, `clamp()` everywhere a length goes; custom properties and `var()` (resolved at cascade time); CSS Color 4/5 (`rgb()`/`hsl()` space syntax, `hwb()`, `lab()`/`lch()`/`oklab()`/`oklch()`, `color-mix()`); gradients kept as a canonical text (`onyx-gradient: URLs`) for the redraw.

- **Properties added:** `row-gap/gap`, the four `border-*-radius`, `box-shadow`, `text-shadow`, `background-size`, `background-clip` (with text), `text-overflow`, `justify-items/justify-self`, `aspect-ratio`, `object-fit/object-position`, `transform/translate/scale/rotate`, the grid properties; logical properties and the CSS3 shorthands mapped to their physical longhands.
- **Rules and selectors:** `@supports`, `@layer`, `@container` (their content kept), `:is()`, `:where()`, `:focus-within`, `:focus-visible`, `:any-link`.
- **Vendor prefixes:** `-webkit-`, `-moz-`, `-ms-`, `-o-` properties whose standard name is known are that property; `-webkit-text-fill-color` is `color`.
- **@font-face:** `format(woff2)`, `unicode-range`, `font-weight` ranges and any weight 1–1000; `css_stylesheet_font_faces()` lists a sheet's rules.
- **Units:** a length is its unit's exact size times its value (upstream rounded the unit to whole pixels first: 1em of a 12.48 px font was 12 px, 1pt was 1px).
- libcss's own selection tests still pass: `make -f tools/tests/netsurf/host.mk libcss-test`.

5. Layout (`content/handlers/html/layout*.c`)

- **Flexbox:** inline children blockified; the automatic minimum size; item contributions as Flexbox 9.9.1 (the larger of content and definite width, clamped by the flex base); `gap`; `justify-content`, `align-content`; `min-height/max-height` with auto height.
- **Grid** (`layout_grid.c`, new): templates, `repeat()`, `fr`, `minmax()`, auto tracks, placement, areas, `auto-fill/auto-fit`, gaps, alignment. A grid container is a `BOX_FLEX/BOX_INLINE_FLEX` box whose display is grid.
- **Lines on their baseline** (CSS 2.1 10.8): the block's strut, text (its font's ascent below the half leading), inline-blocks and buttons (their last / first line's baseline), images (their bottom edge), `vertical-align` (sub, super, middle, text-top, text-bottom, top, bottom, lengths) — NetSurf put each box at the top of the line and a text's baseline three quarters down. `line-height: normal` is the font's spacing (was 1.3 em); a run of lines keeps its fractions of a pixel.
- `transform`'s translation moves a box as a relative offset (paint and hit-testing).
- The space after an inline-block, inline-flex, image or control is kept.
- `<button>` keeps a flex / grid / block box.

6. Painting (`content/handlers/html/redraw.c`, `onyx_paint.c`)

- New plotter operations (`include/netsurf/onyx_paint.h`, `plotters.h`): `onyx_shape` (rounded boxes, rings, blurred shadows, gradients), `onyx_round_clip`, `onyx_text_paint` (text in a gradient); the framebuffer's in `frontends/framebuffer/onyx_paint.c` — anti-aliased by signed distance, Gaussian (erfc) shadows, linear / radial / conic / repeating gradients.
- `border-radius`, `box-shadow` (outer and inset), gradient backgrounds, rounded clips of a box's content; `background-clip: text` paints the descendants' text with the box's background.
- Translucent colours are blended; a translucent fill no longer hides what is under it (`desktop/knockout.c`); `color: transparent` draws nothing.
- **The painting order of positioned boxes** (CSS 2.1 appendix E, simplified): NetSurf painted the boxes in the tree's order, so a menu panel positioned over the page (a header's absolute dropdown, a sticky bar with a `z-index`) was drawn under what follows it. A positioned box (and a flex / grid item with a `z-index`) is now put off while its layer — the page, or the positioned box it is in — is painted, then painted after that layer's in-flow content, sorted by `z-index` (auto: 0), each a layer of its own; a negative `z-index` is painted in place (`html_redraw_layer_z`, `onyx_layer_*`).

- **The box under the pointer** follows the same order (`interaction.c`, `onyx_hit_*`): the last box painted there, then its ancestors' links, controls and titles, root first — a click on an open menu reaches its link, not the page under it. A click on a link's text is the `<a>`'s (NetSurf gave the block's node: a text box has none).

7. JavaScript (QuickJS)

NetSurf's JavaScript was Duktape (ES5) with bindings generated by `nsgenbind`, and the page was never laid out again after a script changed it. It is now **QuickJS** (QuickJS-ng 0.17.0, ES2023: classes, `let/const`, arrow functions, promises, `async/await`, optional chaining...) with the DOM written in JavaScript:

- `quickjs/qjs.c` implements `javascript/js.h` on QuickJS: one runtime per window, one context per document, `js_exec` for the scripts NetSurf runs as it parses. `libdom`'s nodes are wrapped (one JavaScript object per node, for the document's life) and a small set of **natives** is given to the prelude: the tree (parent, children, insert, remove, clone, attributes, text), `setHTML` (hubbub's fragment parser: `innerHTML`), the boxes (a node's rectangle, whether it is displayed, a few computed properties), the window (its scroll, its size), timers, navigation, history, cookies (NetSurf's `urldb`), `document.write` (while parsing), the form controls (`formValue`, `formChecked`: NetSurf's controls, what the user typed) and `submit` (NetSurf sends the form: its method, its encoding).
- `quickjs/dom.js` (compiled in as a C string, `qjs_dom_js.h`, made by the `makefiles`) is the DOM on those natives: `Node`, `Element`, `HTMLElement` and ~60 element classes, `Document`, events (capture, target, bubble; `on*` properties and attributes; `preventDefault`), `querySelector/matches/closest` (its own selector engine: `:is`, `:not`, `:has`, `:nth-*`...), `classList`, `dataset`, `style`, `innerHTML/outerHTML`, `MutationObserver`, `IntersectionObserver`, `ResizeObserver`, `matchMedia`, `getComputedStyle`, `localStorage` (kept: a file per origin next to the cookie file, `ls-<origin>.json`, written a turn after a change; `sessionStorage` in memory), `URL`, `URLSearchParams`, `FormData`, `TextEncoder` / `TextDecoder` (UTF-8 in C: `N.utf8`), timers, `requestAnimationFrame`, `location`, `history`, `navigator`.
- **fetch and XMLHttpRequest**: `fetch` (a promise of a `Response`: `ok`, `status`, `statusText`, `url`, `redirected`, `headers`; `text()`, `json()`, `arrayBuffer()`, `bytes()`, `blob()`, `formData()`, `clone()`; an `AbortSignal` cancels it), `Headers`, `Request`, `Response` (`Response.json`, `.error`, `.redirect`), and `XMLHttpRequest` (`readyState` 1 → 4 with `readystatechange`, `loadstart` / `progress` / `load` / `error` / `abort` / `timeout` / `loadend`, `responseType` `'` / `text` / `json` / `arraybuffer` / `blob`, `setRequestHeader`, `getResponseHeader`, `getAllResponseHeaders`, `timeout`, `abort`). Bodies: a string, `URLSearchParams`, `FormData` (url-encoded), a `Blob`, `bytes` (as text: no NUL). On one native, `request(method, url, body, headers, binary, cb)` (`qjs.c`): NetSurf's low-level cache with `LLCACHE_RETRIEVE_FORCE_FETCH`, the callback on the page's thread (the promises' jobs run after it, a changed DOM laid out again); the requests in flight are cancelled when the document goes. Not yet: synchronous XHR (it runs `async`), streams (`body` is null), multipart bodies, `responseXML`, CORS checks (every origin answers).
- **A changed DOM is laid out again** (`html.c`, `html_script_dom_changed`): once a script is done, NetSurf builds the document's boxes again (`dom_to_box_now`, synchronously) and lays it out — a menu a script opens, a class toggled, nodes added. The old boxes' objects (images) are taken over by the new ones (`html_fetch_object`: no fetch, no flicker), the form controls kept (their text areas, their typed text; a select's options read again), the focus and the iframes moved to the new boxes. A script asking for a rectangle gets the new layout at once (`html_script_layout_now`).

- **The browser's events** reach the scripts (`html_script_event`): `mousedown`, `mouseup`, `click` at the element under the pointer — a click a script prevents neither follows its link nor sends its form —; the pointer's moves (`onyx:hover` from `interaction.c`, every move: `dom.js` makes `mouseover` / `mouseout` with their `relatedTarget`, `mouseenter` / `mouseleave` for each ancestor entered or left, and `mousemove`); **CSS :hover** -- the node under the pointer is kept (`html_content.hover_node`); `libcss` asks `node_is_hover` (`css/select.c`: that node and its ancestors), and when it changes the styles are made again (`html_script_dom_changed`) -- only when the style sheets have `:hover` rules (a `:hover` selector was tried: `uses_hover`); `keydown/keypress/keyup` at the focused control or the document (a key prevented is not typed); `input` (typing; deferred a turn: the text area is not changed under its feet), `change` (checkboxes, radios, a select's menu), `submit` (a submit button, Enter; prevented: not sent); the window's `scroll` (the frontend tells the core: `browser_window_scrolled`), `resize`, `DOMContentLoaded` (the document parsed), `load` (laid out, its images in; `<body onload>`).
- A script runs at most `script_timeout` seconds (NetSurf's option); its recursion is stopped past **4 MB** of stack (a `RangeError`). The Onyx app has an 8 MB stack for it: `stack = 8M` in `SD:/apps/netsurf.app/app.txt`, read by the kernel (`AppStackSize`, `kernel/kernel.cpp`) — the default is 256 KB. `NS_JSDEBUG=1` (the PC bench) prints the scripts' errors and `console.log` on `stderr`.
- **libdom**: a changed `class` attribute updates the element's classes (`element.c`: they were cached when the attribute was made, so a `classList` change did not restyle).
- The Duktape backend (`javascript/duktape`, `user/netsurf/gen/duktape`) is no longer built.

8. Known gaps

- JavaScript: no `canvas`; no streams (`fetch`'s `body`), synchronous XHR (runs `async`), multipart request bodies, `responseXML`; CSS `:active` / `:focus`; a form control outside a form is made again at each layout the scripts cause (its old one leaks).
- SVG (inline `<svg>` and `.svg` images), `opacity`, filters, animations and transitions.
- A face split by `unicode-range` outside Latin-1 falls back to the card's fonts.